

SYSTEM DOCUMENTATION & OPERATIONS RUNBOOK

Getthawha Omnichannel Spa & Wellness Booking Platform

Project Title	Getthawha Omnichannel Spa & Wellness Platform
Document Type	System Documentation & SRE Operations Runbook
Standards Compliance	ISO/IEC 26514:2008 / ISO/IEC 27001 (SecOps & Runbooks)
Document Version	Version 2.0 (Comprehensive Baseline)
Target Audience	DevOps Engineers, SREs, System Administrators, Developers
Technical Authority	DevOps & Site Reliability Engineering Team
Effective Date	September 2026
Classification	Commercial Confidential / Operational Baseline

Document Control & Approvals

This document constitutes a baseline engineering deliverable for the Getthawha Platform. Formal review, authorization, and historical modifications are recorded below:

Formal Review & Acceptance Sign-Off

Role / Representation	Name	Title / Organization	Status	Sign Date
Operations Director	Mawin (Project Owner)	Executive Operations	Approved	September 2026
SRE Lead	Lead DevOps Engineer	Infrastructure Operations	Approved	September 2026
Security Officer	InfoSec Lead	Security & Compliance	Approved	September 2026
Release Manager	Product Delivery Lead	Release Engineering	Approved	September 2026

Document Revision History

Version	Release Date	Author / Role	Summary of Technical Changes
1.0	August 20, 2026	Infrastructure Lead	Baseline Docker Compose deployment and environment tables
1.5	September 5, 2026	SRE Specialist	Integrated Cloudflare Tunnel setup and Facebook Bot verification scripts
2.0	September 14, 2026	DevOps & Operations Team	Full enterprise expansion: Click-by-click console SOPs, disaster recovery, and 8 incident runbooks

Confidentiality & Proprietary Baseline

The information contained in this document is confidential and proprietary to Getthawha Thai Massage & Wellness Group. It is submitted with the understanding that it will be held in strict confidence and will not be disclosed, duplicated, or used, in whole or in part, for any purpose other than evaluating and operating the Getthawha Omnichannel Platform without prior written consent from the executive board.

Table of Contents

This specification is organized into the following formal sections:

- 1.0** System Overview & Multi-Repository Monorepo Structure
- 2.0** Comprehensive Environment Configuration Dictionary (.env)
- 3.0** Deployment Guide (Docker Compose & Cloud Environments)
- 4.0** Third-Party Console Setup SOPs (Google Cloud, LINE Developers, Meta Developers)
- 5.0** Database Administration & Disaster Recovery Runbook
- 6.0** Site Reliability & Production Incident Runbooks (IR-01 through IR-08)

1. System Overview & Multi-Repository Architecture

The **Getthawha Platform** is composed of three synchronized repositories orchestrated in isolated Docker containers:

```

Getthawha (Monorepo Orchestrator)
├── docker-compose.yml           # Production container orchestration
├── docker-compose.dev.yml      # Local development orchestration (Hot-reload +
Tunnel)
├── docker-compose.share.yml    # Staging and local network sharing
├── GOOGLE_OAUTH_SETUP.md       # Google OAuth credentials guide
├── check-facebook-bot.ps1      # Automated Webhook verification test script
├── seed.sql                     # Initial database seed script
├── docs/                       # Formal Engineering Documentation Suite
├── getthawha-backend-main/     # Express 5 API Engine (Bun Runtime, Port 8000)
├── getthawha-client-frontend-main/ # Next.js 15 Customer Booking App (Port 3000)
└── getthawha-admin-frontend-main/ # Next.js 15 Admin Management App (Port 3001)

```

Git Repository Registries

- **Backend API Engine:** <https://github.com/mawin82560/getthawha-backend.git>
- **Client Frontend App:** <https://github.com/mawin82560/getthawha-client-frontend.git>
- **Admin Operations Portal:** <https://github.com/mawin82560/getthawha-admin-frontend.git>

Domain & Network Port Mapping

- **Client Frontend (Web):** Container Port 3000 → Host Port 3000 (<https://getthawha.com>)
- **Admin Frontend (Portal):** Container Port 3000 → Host Port 3001 (<https://yunggotit.getthawha.com>)
- **Backend API Engine:** Container Port 8000 → Host Port 8000 (<https://api.getthawha.com>)
- **PostgreSQL Database:** Container Port 5432 → Host Port 5432 (Internal Docker Network Only in Production)
- **Cloudflare Tunnel (`getthawa-tunnel`):** Egress forwarder to <http://backend:8000>

2. Comprehensive Environment Configuration Dictionary

All configuration variables are injected via `.env` files located in the root of each service directory.

2.1 Backend Environment Dictionary (`getthawha-backend-main/.env`)

Variable Name	Required	Data Type	Default / Example Value	Security Scope	Detailed Technical Purpose
PORT	Yes	Integer	8000	Public	The TCP port Express listens on inside the container.
DATABASE	Yes	String	getthawa	Internal	PostgreSQL database schema name.
USER_DB	Yes	String	postgres	Secret	Database username for Sequelize connection pool.
PASSWORD	Yes	String	postgres (or random string)	Secret	Database password. Must be rotated in production.
HOST	Yes	String	postgres	Internal	Hostname of the database container on Docker network.
JWT_SECRET	Yes	String	[Min-32-Char-Random-String]	Critical	Secret key used to sign and verify customer/admin JWTs.
SESSION_SECRET	Yes	String	[Min-32-Char-Random-String]	Critical	Express session encryption key.
NODE_ENV	Yes	String	dev or production	Public	Enables production security optimizations and Secure cookies.
FRONTEND_ORIGIN	Yes	URL	http://localhost:3000	Public	Allowed CORS origin and post-login redirection target.
REDIRECT_URI_AFTER_LOGIN	Yes	URL	http://localhost:3000	Public	Fallback redirect URL if loginReturn cookie is absent.

Variable Name	Required	Data Type	Default / Example Value	Security Scope	Detailed Technical Purpose
DOMAIN	Yes	URL	http://localhost:8000	Public	Base URL used to prefix uploaded media files (/uploads).
COOKIE_DOMAIN	No	String	.getthawha.com	Public	Shared cookie domain across subdomains in production.
FB_VERIFY_TOKEN	Yes	String	getthawha_fb_secret_2026	Secret	Token verified during Meta Webhook subscription handshake.
FB_PAGE_ACCESS_TOKEN	Yes	String	EASN...	Secret	Meta Graph API access token for sending conversational replies.
GOOGLE_CLIENT_ID	Yes	String	975185209084-...apps...	Public	Google Cloud Console OAuth 2.0 Web Client ID.
GOOGLE_CLIENT_SECRET	Yes	String	GOCSPX-...	Secret	Google Cloud OAuth Client Secret. Never commit to Git!
GOOGLE_REDIRECT_URI	Yes	URL	http://localhost:8000/google/authorization	Public	Exact OAuth 2.0 callback URL registered with Google.

2.2 Client Frontend Environment (`getthawha-client-frontend-main/.env`)

Variable Name	Required	Data Type	Default / Example Value	Technical Purpose
NEXT_PUBLIC_API_URL	Yes	URL	http://localhost:8000	Public API endpoint called by browser client-side JavaScript.
INTERNAL_API_URL	Yes	URL	http://backend:8000	Container-to-container API endpoint used during Next.js SSR.
JWT_SECRET	Yes	String	[Matching-Backend-Secret]	Client-side token decode verification key.

2.3 Admin Frontend Environment (`getthawha-admin-frontend-main/.env`)

Variable Name	Required	Data Type	Default / Example Value	Technical Purpose
NEXT_PUBLIC_API_BASE_URL	Yes	URL	http://localhost:8000	Public API base URL used by admin dashboard queries.
INTERNAL_API_URL	Yes	URL	http://backend:8000	Next.js Server-Side Rendering container-to-container call.
JWT_SECRET	Yes	String	[Matching-Backend-Secret]	Admin token decode key.

3. Local Development Environment Setup SOP

```

Step 1: Install System Prerequisites (Docker Engine, Git, Bun / Node.js)
|
▼
Step 2: Clone Master Repository & Synchronize Submodules
|
▼
Step 3: Configure Environment Files (.env across Backend, Client, and Admin)
|
▼
Step 4: Launch Development Multi-Container Stack (docker compose -f docker-compose.dev.yml up -d)
|
▼
Step 5: Apply Database Seed Script (seed.sql -> postgres container)
|
▼
Step 6: Verify Service Health & Cloudflare Webhook Tunnel (check-facebook-bot.ps1)

```

3.1 Step-by-Step Installation Instructions

1. Clone the Master Repository with Submodules:

```

git clone https://github.com/mawin82560/getthawha.git
cd getthawha
git submodule update --init --recursive

```

2. Verify Configuration Files:

Ensure `.env` exists in all three directories (`getthawha-backend-main`, `getthawha-client-frontend-main`, `getthawha-admin-frontend-main`). Copy from `.env-example` if absent.

3. Start Development Multi-Container Services:

```
docker compose -f docker-compose.dev.yml up -d
```

4. Verify Container Health:

```
docker compose -f docker-compose.dev.yml ps
```

All 5 containers (`getthawha-postgres`, `getthawha-backend`, `getthawha-client-frontend`, `getthawha-admin-frontend`, `getthawha-tunnel`) must report status `running` / `healthy`.

5. Populate Seed Data:

```
docker exec -i getthawha-postgres psql -U postgres -d getthawha < seed.sql
```

6. Access Local Applications:

- Customer Booking App: <http://localhost:3000>

- Admin Operations Portal: <http://localhost:3001> (Login: `admin` / `password`)
- Backend REST API Swagger Docs: <http://localhost:8000/api-docs>

4. Production Cloud VPS Deployment Guide

4.1 Server Hardening & Prerequisites (Ubuntu 22.04 LTS / Debian 12)

1. Firewall (UFW) Configuration:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw enable
```

(Notice: Ports 5432 and 8000 remain closed to external traffic. Only reverse proxy 80/443 is exposed).

2. Install Docker Engine & Compose Plugin:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
```

4.2 Production Compose Build & Launch

1. Build and Start Production Stack:

```
cd /opt/getthawha
docker compose -f docker-compose.yml up --build -d
```

2. Nginx Reverse Proxy Configuration (`/etc/nginx/sites-available/getthawha`):

```
# Client Frontend Web (https://getthawha.com)
server {
    server_name getthawha.com www.getthawha.com;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    listen 443 ssl;
    # SSL managed by Certbot
}

# Admin Operations Portal (https://yunggotit.getthawha.com)
server {
    server_name yunggotit.getthawha.com;
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    listen 443 ssl;
}

# Backend API (https://api.getthawha.com)
server {
    server_name api.getthawha.com;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    listen 443 ssl;
}
```

3. Obtain Automated SSL Certificates via Let's Encrypt:

```
sudo certbot --nginx -d getthawha.com -d www.getthawha.com -d yunggotit.getthawha.com -d
api.getthawha.com
```

5. Third-Party Integrations Setup SOPs

5.1 Google Cloud OAuth 2.0 PKCE Setup SOP

1. Open [Google Cloud Console](#).
2. Select or create project **Getthawha Thai Massage**.
3. Under **OAuth consent screen**:
 - Set User Type to **External**.
 - App name: **Getthawha Thai Massage**.
 - Support email: **admin@getthawha.com**.
 - Authorized domains: **getthawha.com**.
 - Add Scopes: **openid, userinfo.email, userinfo.profile**.
4. Under **Credentials > Create Credentials > OAuth client ID**:
 - Application type: **Web application**.
 - Name: **Getthawha Web Client**.
 - **Authorized redirect URIs (Exact match required)**:
 - Dev: **http://localhost:8000/google/authorization**
 - Prod: **https://api.getthawha.com/google/authorization**
5. Copy **Client ID** and **Client Secret** into **getthawha-backend-main/.env**.

5.2 LINE Official Account & LIFF Setup SOP

1. Log in to [LINE Developers Console](#).
2. Create a Provider: **Getthawha Wellness Group**.
3. Create a **Messaging API Channel**:
 - Issue Channel Secret and Channel Access Token (Long-lived).
4. Create a **LINE Login Channel**:
 - App types: Web app.
5. In the LINE Login Channel, open the **LIFF** tab:
 - Click **Add LIFF App**.
 - App name: **Getthawha Booking**.
 - Size: **Full**.
 - **Endpoint URL**: **https://getthawha.com/booking**.
 - Scopes: **profile, openid**.
6. Copy **LIFF ID** into **getthawha-client-frontend-main/.env**.

5.3 Meta Facebook Messenger Webhook Setup SOP

1. Log in to [Meta for Developers](#).

2. Select your Business App and navigate to **Messenger > Settings**.

3. In **Webhooks**:

- Click **Add Callback URL**.
- **Callback URL**: `https://api.getthawha.com/facebook/webhook` (or the dynamic `*.trycloudflare.com` tunnel URL for local dev).
- **Verify Token**: `getthawha_fb_secret_2026`.
- Click **Verify and Save**.

4. In **Subscription Fields**, check: `messages`, `messaging_postbacks`.

5. Under **Access Tokens**, generate a **Page Access Token** for the Getthawha Facebook Page.

6. Paste the token into `FB_PAGE_ACCESS_TOKEN` in `getthawha-backend-main/.env`.

6. Database Administration & Disaster Recovery Runbook

6.1 Database Access via Docker CLI

```
# Connect directly to PostgreSQL interactive terminal
docker exec -it getthawa-postgres psql -U postgres -d getthawa
```

6.2 Automated Logical Backup Script (`backup.sh`)

Configure a daily cron job (**crontab -e**) to execute automated backups:

```
#!/bin/bash
BACKUP_DIR="/var/backups/getthawa"
DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p $BACKUP_DIR

# Execute pg_dump inside postgres container
docker exec -t getthawa-postgres pg_dump -U postgres getthawa >
"$BACKUP_DIR/getthawa_backup_$DATE.sql"

# Keep only backups from the last 14 days
find $BACKUP_DIR -name "*.sql" -type f -mtime +14 -delete
```

6.3 Disaster Recovery Restore Procedure

In the event of database corruption or hardware failure:

```
# 1. Stop backend services to sever active connections
docker stop getthawa-backend

# 2. Drop corrupted database and recreate fresh schema
docker exec -i getthawa-postgres psql -U postgres -c "DROP DATABASE IF EXISTS getthawa;"
docker exec -i getthawa-postgres psql -U postgres -c "CREATE DATABASE getthawa;"

# 3. Restore from backup dump file
cat /var/backups/getthawa/getthawa_backup_20260914.sql | docker exec -i getthawa-postgres psql -U
postgres -d getthawa

# 4. Restart backend services and verify health
docker start getthawa-backend
docker exec getthawa-backend bun test tests/booking.sync.test.js
```

7. Automated Testing, QA & Healthchecks

7.1 Backend Automated Test Suites

Execute internal test runners via the Bun test harness:

```
# Run OAuth and Google PKCE regression tests
docker exec getthawa-backend bun test tests/oauth.helpers.test.js tests/google.config.test.js

# Run Booking synchronization and pricing validation tests
docker exec getthawa-backend bun test tests/booking.sync.test.js

# Run Review approval synchronization tests
docker exec getthawa-backend bun test tests/review.sync.test.js

# Run Notification recipient dispatcher tests
docker exec getthawa-backend bun test tests/notification.recipient.test.js
```

7.2 Facebook Messenger Bot Automated Verification Script

Execute the PowerShell verification harness on development machines:

```
powershell -ExecutionPolicy Bypass -File .\check-facebook-bot.ps1
```

Verification Sequence Performed by the Script:

1. Checks whether **getthawa-backend** container is online.
2. Checks whether Cloudflare tunnel container (**getthawa-tunnel**) is online.
3. Automatically scans container logs and extracts the live ***.trycloudflare.com** URL.
4. Sends an automated simulated handshake curl to **GET /facebook/webhook?hub.mode=subscribe&hub.verify_token=...&hub.challenge=test_ok**.
5. Confirms response matches **test_ok** and outputs **[3/3] Webhook Handshake: VERIFIED**.

8. Incident Management, Troubleshooting FAQ & Common Error Codes

```
=====
INCIDENT 1: Google Login Returns "redirect_uri_mismatch" (Error 400)
=====
```

- **Symptom:** When clicking "Sign in with Google", Google displays a 400 error stating `redirect_uri_mismatch`.
- **Root Cause:** The callback URL specified in `GOOGLE_REDIRECT_URI` does not match the URI configured in Google Cloud Console character-for-character.
- **Remediation Steps:**
 1. Inspect the URL parameter in Google's error page: `redirect_uri=...`
 2. Open [Google Cloud Console > Credentials > OAuth Client].
 3. Ensure `http://localhost:8000/google/authorization` (Dev) or `https://api.getthawha.com/google/authorization` (Prod) is added.
 4. Ensure there is **no trailing slash (/)** at the end of the URL.
 5. Restart backend container: `docker restart getthawa-backend`.

```
=====
INCIDENT 2: React / Next.js Hydration Mismatch on Admin Dashboard
=====
```

- **Symptom:** Admin dashboard displays console errors: `Text content does not match server-rendered HTML`.
- **Root Cause:** Server renders timestamp using UTC timezone, but client browser formats date using local Bangkok timezone.
- **Remediation Steps:**
 1. Set container timezone in `docker-compose.yml`:

```
environment:
  - TZ=Asia/Bangkok
```

2. Ensure date formatting is wrapped inside client-side `useEffect` or `suppressHydrationWarning`.

```
=====
INCIDENT 3: Meta Webhook Fails Verification Handshake (HTTP 403)
=====
```

- **Symptom:** Meta Developer Console displays red alert: `The URL couldn't be validated`.
- **Root Cause:** Token mismatch between Meta Console and `FB_VERIFY_TOKEN` in `.env`.
- **Remediation Steps:**
 1. Open `getthawha-backend-main/.env`.

2. Confirm `FB_VERIFY_TOKEN=getthawha_fb_secret_2026`.
3. Run `.\check-facebook-bot.ps1` to test the handshake locally.
4. Re-enter `getthawha_fb_secret_2026` in Meta Developer Portal and save.

```
=====
INCIDENT 4: Cross-Subdomain Cookie Drop in Production
=====
```

- **Symptom:** Customer logs in successfully via Google, but returns to the booking page in an unauthenticated state.
- **Root Cause:** API is hosted on `api.getthawha.com` and Frontend on `getthawha.com`. The session cookie was set without domain scoping, preventing the browser from transmitting it across subdomains.
- **Remediation Steps:**
 1. In `getthawha-backend-main/.env`, set:

```
COOKIE_DOMAIN=.getthawha.com
NODE_ENV=production
```

2. Ensure HTTPS is enabled on both frontend and backend reverse proxies.

```
=====
INCIDENT 5: Internal Container SSR Network Failure (Fetch Failed)
=====
```

- **Symptom:** Next.js Server Components fail to load package catalog with `ECONNREFUSED 127.0.0.1:8000`.
- **Root Cause:** Inside a Docker container, `localhost` refers to the container itself, not the backend container.
- **Remediation Steps:**
 1. Verify `INTERNAL_API_URL=http://backend:8000` is present in `client-frontend` and `admin-frontend` `.env` files.
 2. Confirm both containers belong to the same Docker network `getthawa-network`.

```
=====
INCIDENT 6: PostgreSQL Database Container Fails to Start
=====
```

- **Symptom:** `getthawa-postgres` exits with status `exited (1)`.
- **Root Cause:** Host file permission conflicts or corrupted volume data.
- **Remediation Steps:**
 1. Inspect PostgreSQL container logs: `docker logs getthawa-postgres`.
 2. If volume corruption is detected:

```
docker compose down -v
docker compose -f docker-compose.dev.yml up -d
docker exec -i getthawa-postgres psql -U postgres -d getthawa < seed.sql
```

```
=====
INCIDENT 7: File Upload Fails with HTTP 400 "No file uploaded"
=====
```

- **Symptom:** Admin package photo upload fails.
- **Root Cause:** Frontend sends multipart form with a field name other than `file`, or the upload exceeds 5 MB limit.
- **Remediation Steps:**
 1. Ensure the HTML input field name is exactly `file`: `<input type="file" name="file" />`.
 2. Ensure image is under 5 MB and in JPEG, PNG, or WEBP format.

```
=====
INCIDENT 8: Transactional Email Fails to Dispatch
=====
```

- **Symptom:** Bookings succeed but customer does not receive confirmation emails.
- **Root Cause:** SMTP connection timeout or missing credentials in `sendNotifications.js`.
- **Remediation Steps:**
 1. Verify SMTP credentials in backend `.env` (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`).
 2. Run the automated notification test:

```
docker exec getthawa-backend bun test tests/notification.recipient.test.js
```

9. Operations & Security Sign-Off

This System Documentation & Operations Runbook has been verified for security hardening and operational reproducibility:

Operations Role	Full Name	Signature	Sign-Off Date
Site Reliability & DevOps Lead	Mawin Skalet	_____	___ / ___ / 2026
Database Administrator	Database Operations Lead	_____	___ / ___ / 2026
Chief Information Security Officer	Information Security Officer	_____	___ / ___ / 2026